

Operating System Experiments: Aims and Codes

Experiment 1

Aim: Explore usage of basic Linux Commands and system calls for file, directory and process management. (e.g.: mkdir, cd, cat, ls, chmod, chown, ps, awk, grep, etc.)

Working Code (BASH):

```
#!/bin/bash
# Basic Linux Commands Demonstration

echo "1. Directory Management:"
mkdir test_dir
cd test_dir || exit
pwd

echo -e "\n2. File Management & Text Processing:"
echo "Linux is great. Linux is open source." > sample.txt
cat sample.txt
echo "Searching for 'Linux':"
grep -i "linux" sample.txt
echo "Word count:"
awk '{print "Words in line:", NF}' sample.txt

echo -e "\n3. Permissions:"
chmod 755 sample.txt
ls -l sample.txt

echo -e "\n4. Process Management:"
ps -eo pid,comm --sort=-%mem | head -n 5

# Clean up
cd ..
rm -rf test_dir
echo "Experiment 1 Completed."
```

Experiment 2

Aim: Write shell scripts to display OS version, release number, kernel version, current logged in user, log name, current shell, home directory, OS type, path setting, working directory, top 10 processes in descending order, and processes with highest memory usage.

Working Code (BASH):

```
#!/bin/bash
# System Information Shell Script

echo "--- System Information ---"
echo "Kernel Release Number : $(uname -r) "
echo "Kernel Version       : $(uname -v) "
echo "Operating System Type : $OSTYPE"
echo "Current User          : $(whoami) "
echo "Logname                : $(logname 2>/dev/null || echo $USER) "
echo "Current Shell          : $SHELL"
echo "Home Directory         : $HOME"
echo "Current Path Setting   : $PATH"
echo "Current Working Dir    : $(pwd) "
echo ""

echo "--- Top 10 Processes (CPU Usage) ---"
ps -eo pid,comm,%cpu --sort=-%cpu | head -n 11
echo ""

echo "--- Processes with Highest Memory Usage ---"
ps -eo pid,comm,%mem --sort=-%mem | head -n 11
```

Experiment 3

Aim: Create a child process in Linux using fork() system call. From the child process obtain the process ID of both child and parent by using getpid() and getppid() system calls. Explore wait() before termination of process.

Working Code (C):

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Starting Process Creation...\n");
    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child Process
        printf("CHILD PROCESS:\n");
        printf("My Process ID (Child) : %d\n", getpid());
        printf("My Parent's ID (Parent) : %d\n", getppid());
    }
    else {
        // Parent Process
        printf("PARENT PROCESS:\n");
        printf("My Process ID (Parent) : %d\n", getpid());
        printf("My Child's ID : %d\n", pid);

        // Wait for child process to terminate
        wait(NULL);
        printf("Child process terminated. Parent exiting.\n");
    }

    return 0;
}
```

Experiment 4

Aim: Process Management: Scheduling - Write a program to demonstrate the concept of non-preemptive scheduling algorithms (FCFS and SJF).

Working Code (CPP):

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Process {
    int id;
    int bt; // Burst time
    int wt; // Waiting time
    int tat; // Turnaround time
};

bool compareBT(Process a, Process b) {
    return a.bt < b.bt;
}

void calculateTimes(vector<Process>& p) {
    p[0].wt = 0;
    p[0].tat = p[0].bt;
    float total_wt = 0, total_tat = p[0].tat;

    for (size_t i = 1; i < p.size(); i++) {
        p[i].wt = p[i - 1].wt + p[i - 1].bt;
        p[i].tat = p[i].wt + p[i].bt;
        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }

    cout << "\nPID\tBurst Time\tWaiting Time\tTurnaround Time\n";
    for (auto proc : p) {
        cout << proc.id << "\t" << proc.bt << "\t\t" << proc.wt << "\t\t" <<
proc.tat << "\n";
    }
    cout << "\nAverage Waiting Time: " << total_wt / p.size() << "\n";
}

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        cout << "Enter burst time for P" << i + 1 << ": ";
    }
}
```

```
        cin >> p[i].bt;
    }

    cout << "\n--- FCFS Scheduling ---";
    calculateTimes(p);

    cout << "\n--- SJF (Non-Preemptive) Scheduling ---";
    vector<Process> sjf_p = p;
    sort(sjf_p.begin(), sjf_p.end(), compareBT);
    calculateTimes(sjf_p);

    return 0;
}
```

Experiment 5

Aim: Process Management: Scheduling - Write a program to demonstrate the concept of preemptive scheduling algorithms (Preemptive Priority and Round Robin).

Working Code (C):

```
#include <stdio.h>

void roundRobin(int n, int bt[], int tq) {
    int rt[n], wt[n], tat[n];
    int time = 0, remain = n;
    float avg_wt = 0, avg_tat = 0;

    for (int i = 0; i < n; i++) {
        rt[i] = bt[i];
    }

    printf("\n--- Round Robin Scheduling ---\n");
    for (int i = 0; remain != 0; ) {
        if (rt[i] <= tq && rt[i] > 0) {
            time += rt[i];
            rt[i] = 0;
            remain--;
            wt[i] = time - bt[i];
            tat[i] = time;
            avg_wt += wt[i];
            avg_tat += tat[i];
        } else if (rt[i] > 0) {
            rt[i] -= tq;
            time += tq;
        }

        if (i == n - 1) i = 0;
        else i++;
    }

    printf("PID\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    printf("Avg Waiting Time: %.2f\n", avg_wt / n);
}

void preemptivePriority(int n, int at[], int bt[], int pr[]) {
    int rt[n], wt[n], tat[n];
    int time = 0, completed = 0, highest, pos;
    float avg_wt = 0, avg_tat = 0;

    for (int i = 0; i < n; i++) rt[i] = bt[i];

    printf("\n--- Preemptive Priority Scheduling ---\n");
    while (completed < n) {
```

```

    highest = 9999;
    pos = -1;

    for (int i = 0; i < n; i++) {
        if (at[i] <= time && rt[i] > 0 && pr[i] < highest) {
            highest = pr[i];
            pos = i;
        }
    }

    if (pos == -1) {
        time++;
        continue;
    }

    rt[pos]--;
    time++;

    if (rt[pos] == 0) {
        completed++;
        wt[pos] = time - bt[pos] - at[pos];
        if (wt[pos] < 0) wt[pos] = 0;
        tat[pos] = time - at[pos];
        avg_wt += wt[pos];
        avg_tat += tat[pos];
    }
}

printf("PID\tAT\tBT\tPR\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], wt[i],
tat[i]);
}
printf("Avg Waiting Time: %.2f\n", avg_wt / n);
}

int main() {
    int bt[] = {3, 3, 3};
    int tq = 2;
    roundRobin(3, bt, tq);

    int at2[] = {0, 2, 3};
    int bt2[] = {7, 4, 6};
    int pr2[] = {2, 1, 6}; // Lower number = Higher priority
    preemptivePriority(3, at2, bt2, pr2);

    return 0;
}

```

Experiment 6

Aim: Process Management: Synchronization - Write a C program to implement solution of Producer-Consumer problem through Semaphores.

Working Code (C):

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for(int i = 0; i < 5; i++) {
        int item = rand() % 100;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d at position %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    pthread_exit(NULL);
}

void* consumer(void* arg) {
    for(int i = 0; i < 5; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d from position %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
}
```

```
        pthread_exit(NULL);
    }

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Experiment 7

Aim: Process Management: Deadlock - Write a program to demonstrate the concept of deadlock avoidance through Banker's Algorithm.

Working Code (C):

```
#include <stdio.h>

int main() {
    int n, m;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m], avail[m];

    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("\nEnter Max Matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("\nEnter Available Resources:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    // Calculate Need Matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    int finish[n], safeSeq[n], count = 0;
    for (int i = 0; i < n; i++) finish[i] = 0;

    while (count < n) {
        int found = 0;
        for (int i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int flag = 0;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        flag = 1;
                        break;
                    }
                }
            }
        }
    }
```

```

    }

    if (flag == 0) {
        for (int k = 0; k < m; k++)
            avail[k] += alloc[i][k];

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if (found == 0) {
    printf("\nSystem is NOT in a safe state (Deadlock possible).\n");
    return 0;
}

printf("\nSystem is in a SAFE state.\nSafe sequence is: ");
for (int i = 0; i < n; i++)
    printf("P%d ", safeSeq[i]);
printf("\n");

return 0;
}

```

Experiment 8

Aim: Memory Management - Write a program to demonstrate the concept of dynamic partitioning placement algorithms (First Fit, Best Fit, Worst Fit).

Working Code (C):

```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int alloc[n], bSize[m];
    for (int i = 0; i < n; i++) alloc[i] = -1;
    for (int i = 0; i < m; i++) bSize[i] = blockSize[i];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (bSize[j] >= processSize[i]) {
                alloc[i] = j;
                bSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\nPID\tProc Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t", i + 1, processSize[i]);
        if (alloc[i] != -1) printf("%d\n", alloc[i] + 1);
        else printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int alloc[n], bSize[m];
    for (int i = 0; i < n; i++) alloc[i] = -1;
    for (int i = 0; i < m; i++) bSize[i] = blockSize[i];

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (bSize[j] >= processSize[i]) {
                if (bestIdx == -1 || bSize[j] < bSize[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            alloc[i] = bestIdx;
            bSize[bestIdx] -= processSize[i];
        }
    }

    printf("\nBest Fit Allocation:\nPID\tProc Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
```

```

        printf("%d\t%d\t\t", i + 1, processSize[i]);
        if (alloc[i] != -1) printf("%d\n", alloc[i] + 1);
        else printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int alloc[n], bSize[m];
    for (int i = 0; i < n; i++) alloc[i] = -1;
    for (int i = 0; i < m; i++) bSize[i] = blockSize[i];

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (bSize[j] >= processSize[i]) {
                if (worstIdx == -1 || bSize[j] > bSize[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            alloc[i] = worstIdx;
            bSize[worstIdx] -= processSize[i];
        }
    }

    printf("\nWorst Fit Allocation:\nPID\tProc Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t\t", i + 1, processSize[i]);
        if (alloc[i] != -1) printf("%d\n", alloc[i] + 1);
        else printf("Not Allocated\n");
    }
}

int main() {
    int blockSize[] = {6, 12, 25, 20};
    int processSize[] = {10, 20, 5, 15};
    int m = sizeof(blockSize) / sizeof(blockSize[0]);
    int n = sizeof(processSize) / sizeof(processSize[0]);

    firstFit(blockSize, m, processSize, n);
    bestFit(blockSize, m, processSize, n);
    worstFit(blockSize, m, processSize, n);

    return 0;
}

```

Experiment 9

Aim: Memory Management: Virtual Memory - Write a program in C to demonstrate the concept of page replacement policies for handling page faults (FIFO, LRU).

Working Code (C):

```
#include <stdio.h>

void fifo(int pages[], int n, int frames) {
    int frame[frames], front = 0, faults = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");
    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            frame[front] = pages[i];
            front = (front + 1) % frames;
            faults++;
            printf("Page %d -> Page Fault | Frames: ", pages[i]);
        } else {
            printf("Page %d -> No Fault    | Frames: ", pages[i]);
        }

        for (int j = 0; j < frames; j++) {
            if (frame[j] != -1) printf("%d ", frame[j]);
            else printf("- ");
        }
        printf("\n");
    }
    printf("Total Page Faults (FIFO): %d\n", faults);
}

void lru(int pages[], int n, int frames) {
    int frame[frames], last_used[frames], faults = 0;
    for (int i = 0; i < frames; i++) {
        frame[i] = -1;
        last_used[i] = -1;
    }

    printf("\n--- LRU Page Replacement ---\n");
    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
```

```

        found = 1;
        last_used[j] = i; // update time
        break;
    }
}

if (!found) {
    int lru_idx = 0, min_time = last_used[0];
    for (int j = 1; j < frames; j++) {
        if (last_used[j] < min_time) {
            min_time = last_used[j];
            lru_idx = j;
        }
    }
    frame[lru_idx] = pages[i];
    last_used[lru_idx] = i;
    faults++;
    printf("Page %d -> Page Fault | Frames: ", pages[i]);
} else {
    printf("Page %d -> No Fault    | Frames: ", pages[i]);
}

for (int j = 0; j < frames; j++) {
    if (frame[j] != -1) printf("%d ", frame[j]);
    else printf("- ");
}
printf("\n");
}
printf("Total Page Faults (LRU): %d\n", faults);
}

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3};
    int n = 12, frames = 3;

    fifo(pages, n, frames);
    lru(pages, n, frames);

    return 0;
}

```

Experiment 10

Aim: File Management & I/O Management - Write a C program to simulate File allocation strategies: sequential, indexed and linked.

Working Code (C):

```
#include <stdio.h>
#include <stdlib.h>

void sequential(int file_size, int block_size) {
    int blocks = (file_size + block_size - 1) / block_size;
    int start = 5; // Simulating user input for starting block

    printf("\n--- Sequential Allocation ---\n");
    for (int i = 0; i < blocks; i++) {
        printf("Block %d -> Disk Block %d\n", i, start + i);
    }
}

void linked(int file_size, int block_size) {
    int blocks = (file_size + block_size - 1) / block_size;
    int arr[] = {2, 5, 8, 11, 14, 17, 20, 23, 26, 29}; // Sample random disk
    blocks

    printf("\n--- Linked Allocation ---\n");
    for (int i = 0; i < blocks - 1; i++) {
        printf("Disk Block %d -> Next Block %d\n", arr[i], arr[i + 1]);
    }
    printf("Disk Block %d -> NULL\n", arr[blocks - 1]);
}

void indexed(int file_size, int block_size) {
    int blocks = (file_size + block_size - 1) / block_size;
    int index_block = 5;
    int arr[] = {3, 7, 9, 12, 18, 23, 29, 34, 42, 48}; // Sample scattered
    blocks

    printf("\n--- Indexed Allocation ---\n");
    printf("Index Block: %d\n", index_block);
    for (int i = 0; i < blocks; i++) {
        printf("Index[%d] -> Disk Block %d\n", i, arr[i]);
    }
}

int main() {
    int file_size = 100;
    int block_size = 10;

    printf("Simulating File Allocation Strategies (File Size = %d, Block Size = %d)\n", file_size, block_size);

    sequential(file_size, block_size);
```

```
    linked(file_size, block_size);  
    indexed(file_size, block_size);  
  
    return 0;  
}
```

